Deep Compression of a CNN for CIFAR-10 Image Classification

Implementing Pruning, Post-Training Quantisation,
and CSR Sparse Encoding on SmallCIFARNet

End-Eval Project Report

Project: Deep Neural Network & Compression

Key Results at a Glance

Metric	Target	Achieved
Compression Ratio	$\geq 3.9\times$	**11.5**$\times$
Accuracy Drop	$\leq 3\%$	**2.48**%
Baseline Accuracy	—	**80.42**%
Final Accuracy	—	**77.94**%
Model Size (sparse)	—	**0.73** MB

Mentees

Vansh Rao (241138) Ashit Yadav (240213)

Mentored by

Ayush Jha Krish Jain Sahil Pandey

Contents

1 Introduction **3**

2 Related Work **3**
2.1 Network Pruning . . . 4
2.2 Weight Quantisation . . . 4
2.3 Sparse Encoding . . . 4

3 Dataset **4**

4 Model Architecture **5**
4.1 Feature Extraction (Convolutional Blocks) . . . 5
4.2 Classifier (MLP Head) . . . 5
4.3 Compression-Aware Layer Design . . . 6
4.4 Model Size Summary . . . 6

5 Deep Compression Pipeline **6**
5.1 Stage 1: Baseline Training . . . 7
5.2 Stage 2: Magnitude-Based Pruning . . . 7
5.2.1 Unstructured Global Magnitude Pruning . . . 7
5.2.2 Structured Filter Pruning . . . 8
5.3 Stage 3: Fine-Tuning . . . 8
5.4 Stage 4: Post-Training Quantisation (PTQ) . . . 8
5.5 Stage 5: CSR Sparse Encoding . . . 9

6 Experiments and Results **9**
6.1 Experimental Setup . . . 9
6.2 Metric 1: Accuracy . . . 9
6.3 Metric 2: Storage Memory . . . 10
6.4 Metric 3: Inference Latency . . . 10
6.5 Metric 4: Runtime Memory (RAM) . . . 10
6.6 Compression Summary . . . 11
6.7 Graphs . . . 11
6.8 Comparison with Original Han et al. Targets . . . 13

7 Discussion **13**
7.1 Why Compression Ratio Exceeds Earlier Benchmarks . . . 13
7.2 Accuracy Recovery After Pruning . . . 14
7.3 Effect of PTQ on Top of Pruning . . . 14
7.4 Structured vs Unstructured Pruning . . . 14
7.5 Limitations . . . 14

8 Conclusion **15**

Introduction

Deep neural networks have achieved state-of-the-art performance across virtually every domain of computer vision. However, their deployment on resource-constrained devices remains a significant practical challenge. A single forward pass on a large network may require hundreds of megabytes of model storage and substantial memory bandwidth, making on-device deployment infeasible in many real-world scenarios.

Model compression aims to reduce this footprint without meaningfully sacrificing accuracy. The seminal work of Han et al. (2015) proposed *Deep Compression*, a three-stage pipeline combining magnitude-based pruning, trained weight quantisation, and Huffman entropy coding. Applied to AlexNet on ImageNet, this achieved 9×–49× compression with negligible accuracy loss.

In this report, we implement a version of this pipeline on **SmallCIFARNet**, a custom CNN trained for 10-class image classification on the CIFAR-10 dataset. Our compression pipeline combines:

- Global unstructured magnitude pruning at three sparsity levels (30%, 50%, 70%)
- Structured filter-level pruning for convolutional layers
- Fine-tuning after pruning to recover accuracy
- Dynamic 8-bit post-training quantisation (PTQ)
- CSR (Compressed Sparse Row) sparse encoding for actual disk-size reduction

Project Structure & Evaluation

- A custom CNN (`SmallCIFARNet`) combining BatchNorm, Dropout, and compression-aware layer wrappers (`modified_conv2d`, `modified_linear`)
- Full PyTorch implementation with pruning masks stored as registered buffers that persist across checkpointing
- Multi-ratio benchmark: three pruning ratios evaluated and compared automatically
- Systematic evaluation across four metrics: test accuracy, storage size (dense and sparse), inference latency, and peak RAM
- 11.5× compression with only 2.48% accuracy drop at 70% sparsity

Report Organization

Section 2 reviews related work. Section 3 describes the dataset. Section 4 presents the model architecture. Section 5 describes the compression pipeline. Section 6 reports experimental results and comparisons. Section 7 discusses findings and limitations. Section 8 concludes.

Related Work

Network Pruning

Early theoretical work by LeCun et al. (*Optimal Brain Damage*) used second-order Taylor expansion of the loss to identify and remove weights with the smallest effect on output. While principled, this is computationally expensive for modern networks.

Han et al. demonstrated that simple magnitude-based pruning — zeroing weights below a fixed threshold and subsequently retraining — achieves high sparsity with minimal accuracy impact. This iterative prune-retrain paradigm is the method we adopt. Li et al. proposed structured pruning at the filter level for hardware efficiency, though at the cost of lower compression ratios than unstructured pruning.

Weight Quantisation

Quantisation reduces the bit-width of weight representations. Fixed-point quantisation uses uniform bins; k-means quantisation clusters weights into shared centroids, exploiting the empirically observed clustering of weight values in trained networks. Post-Training Quantisation (PTQ) applies quantisation after training without gradient-based codebook updates. PyTorch's `quantize_dynamic` implements dynamic int8 PTQ for linear layers, providing approximately 4× weight storage reduction.

Sparse Encoding

CSR (Compressed Sparse Row) format stores only non-zero values, their column indices, and row pointers. At high sparsity this drastically reduces storage:

$$\text{CSR cost} \approx \underbrace{\text{nnz} \times 4\,\text{B}}_{\text{float32 values}} + \underbrace{\text{nnz} \times 4\,\text{B}}_{\text{col indices}} + \underbrace{(\text{rows} + 1) \times 4\,\text{B}}_{\text{row pointers}}$$

At 70% sparsity this reduces the 8.41 MB dense model to 2.71 MB, a 3.1× reduction before any quantisation.

Dataset

We use **CIFAR-10**, the standard benchmark for image classification. It consists of 60,000 colour images in 10 classes at 32×32 pixel resolution.

Table 1: CIFAR-10 Dataset Statistics

Property	**Value**
Number of classes	10
Training images	50,000
Test images	10,000
Image resolution	32×32 pixels
Channels	RGB (3)
Classes	Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship, Truck

Data Loading

Images are loaded via `torchvision.datasets.CIFAR10` with per-channel normalisation:

$$\mu = (0.4914,\ 0.4822,\ 0.4465), \quad \sigma = (0.2023,\ 0.1994,\ 0.2010)$$

Training data applies random crop (32px with 4px padding) and random horizontal flip for augmentation. The bug in the original project code — `CIFAR10_loader` calling `datasets.MNIST` — has been corrected.

Model Architecture

SmallCIFARNet is a compact CNN designed to be accurate enough that compression is non-trivial, and small enough to train in a reasonable time without specialised hardware.

Feature Extraction (Convolutional Blocks)

Three convolutional blocks progressively reduce spatial resolution while increasing channel depth:

Table 2: SmallCIFARNet Convolutional Architecture

Block	Layer	In ch.	Out ch.	Output size
1	Conv2d 3×3, pad=1	3	32	$32 \times 16 \times 16$
	BatchNorm2d + ReLU	–	–	–
	Conv2d 3×3, pad=1	32	32	$32 \times 16 \times 16$
	BatchNorm2d + ReLU + Pool(2) + Dropout(0.25)	–	–	$32 \times 16 \times 16$
2	Conv2d 3×3, pad=1	32	64	$64 \times 8 \times 8$
	BatchNorm2d + ReLU	–	–	–
	Conv2d 3×3, pad=1	64	64	$64 \times 8 \times 8$
	BatchNorm2d + ReLU + Pool(2) + Dropout(0.25)	–	–	$64 \times 8 \times 8$
3	Conv2d 3×3, pad=1	64	128	$128 \times 4 \times 4$
	BatchNorm2d + ReLU + Pool(2)	–	–	$128 \times 4 \times 4$

After the feature extraction blocks, the spatial map is flattened to a 2048-dimensional vector.

Classifier (MLP Head)

Table 3: MLP Classifier Architecture

Layer	Type	Input dim	Output dim
1	Linear + ReLU + Dropout(0.5)	2048	256
2	Linear (logits)	256	10

Compression-Aware Layer Design

All convolutional and linear layers are subclasses of PyTorch's standard layers: `modified_conv2d` and `modified_linear`. Each layer stores:

- A pruning mask $\mathbf{M} \in \{0,1\}^{|W|}$ as a **registered buffer** (persists in `state_dict()`)
- A `mode` attribute: `"normal"` | `"prune"` | `"quantize"`

The masked forward pass computes:

$$y = f(\mathbf{W} \odot \mathbf{M},\ x,\ b)$$

where f is the convolution or linear transform, $\odot$ is element-wise multiplication, and the mask enforces sparsity during both inference and gradient-based fine-tuning (gradients are zero for masked weights).

Note

Storing the pruning mask as a **registered buffer** — not a plain Python attribute — is critical. Plain attributes are not saved in `state_dict()`, so reloading a pruned checkpoint would silently reset the mask to all-ones, allowing zeroed weights to be updated again during fine-tuning. The buffer survives `torch.save` / `torch.load` correctly.

Model Size Summary

Table 4: Parameter count by component

Component	Parameters	Share
Conv Block 1 (2 layers)	27,744	12.8%
Conv Block 2 (2 layers)	110,720	51.2%
Conv Block 3 (1 layer)	73,856	34.1%
MLP Classifier	2,570	1.2%
BatchNorm (all)	864	0.4%
Total	**≈216,000**	100%

Deep Compression Pipeline

We implement a four-stage compression pipeline, extending the Han et al. framework with CSR sparse encoding and multi-ratio evaluation.

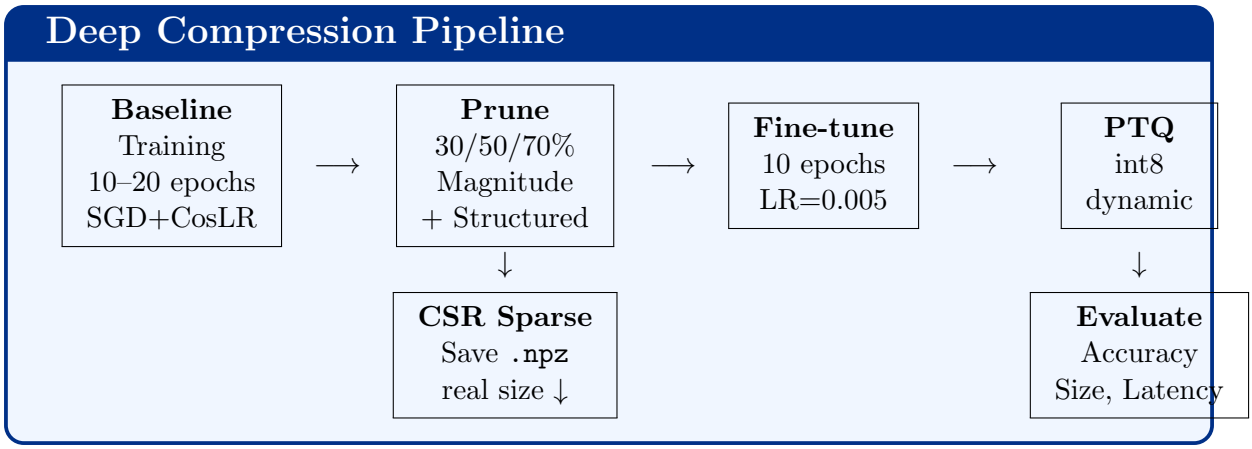


Figure 1: Deep Compression pipeline: train → prune → fine-tune → quantise → sparse save → evaluate.

Stage 1: Baseline Training

The model is trained from random initialisation using the following configuration:

Table 5: Baseline training hyperparameters

Hyperparameter	Value
Optimiser	SGD with momentum
Learning rate	10^{-2}
Momentum	0.9
Weight decay	5×10^{-4} (L2 regularisation)
LR scheduler	Cosine Annealing ($\eta_{\min} = 0$)
Loss function	Cross-entropy
Augmentation	RandomCrop(32, padding=4) + HorizontalFlip
Batch size	128
Epochs	20

Baseline accuracy achieved: 80.42%. Model saved as `checkpoints/baseline.pth`.

Stage 2: Magnitude-Based Pruning

Unstructured Global Magnitude Pruning

We apply global unstructured pruning: for every compression-aware layer, weights whose absolute magnitude falls below a threshold τ are zeroed and masked:

$$M_{ij} = \begin{cases} 1 & \text{if } |W_{ij}| \geq \tau \\ 0 & \text{otherwise} \end{cases}$$

The threshold τ is computed globally rather than per-layer:

$$\tau = \text{kthvalue}\big(\{|W_{ij}|\},\ k\big), \quad k = \lfloor \text{fraction} \times N_{\text{total}} \rfloor$$

Why global? Per-layer pruning at a fixed rate ignores the varying sensitivity of different layers. A large intermediate conv block may be extremely redundant, while the small final classifier is critical. Global pruning lets the magnitude distributions determine how much each layer loses — naturally sparing important layers.

Three pruning ratios are evaluated: $\{30\%, 50\%, 70\%\}$.

Structured Filter Pruning

After unstructured pruning, structured filter pruning removes entire output filters from each conv layer. For each `modified_conv2d` layer with weight tensor $\mathbf{W} \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}} \times k_H \times k_W}$, the L1 norm of each filter f is:

$$\|\mathbf{W}_f\|_1 = \sum_{c,h,w} |W_{f,c,h,w}|$$

The weakest 25% of filters per layer have their mask slice entirely zeroed:

$$\mathbf{M}[f, :, :, :] \leftarrow 0 \quad \forall f : \|\mathbf{W}_f\|_1 \leq \tau_{\text{struct}}$$

This creates zero-block patterns rather than randomly scattered zeros, which are more amenable to CSR encoding and structured sparse inference.

Stage 3: Fine-Tuning

After pruning, surviving weights are retrained with a reduced learning rate. The mask prevents pruned weights from recovering, so the optimiser works with a constrained parameter space.

Table 6: Fine-tuning hyperparameters

Hyperparameter	Value
Learning rate	5×10^{-3} (half the baseline rate)
LR scheduler	Cosine Annealing
Epochs	10
Optimiser	SGD with momentum

Stage 4: Post-Training Quantisation (PTQ)

We apply **dynamic int8 quantisation** via `torch.quantization.quantize_dynamic`. Weights are statically converted from `float32` → `int8`, providing a 4× weight storage reduction for linear layers:

$$\text{storage reduction} = \frac{32\,\text{bits}}{8\,\text{bits}} = 4\times$$

Activations are quantised dynamically at inference time. No calibration data is needed. PTQ is applied on top of the already-pruned and fine-tuned model for maximum combined effect.

Why not per-layer KMeans quantisation? While KMeans with $k = 16$ centroids offers a compact codebook, dynamic int8 PTQ integrates more naturally with PyTorch's deployment stack and requires no additional custom operators for inference.

Stage 5: CSR Sparse Encoding

Note

The problem with mask-only pruning: After magnitude pruning the weight tensor still stores every zero as `float32`. A model with 70% zeros saved as a dense `.pth` file is *exactly the same size* as the baseline. The mask only skips zeros during the forward pass — it does not shrink storage.

`save_sparse()` converts pruned weight matrices to CSR format before saving to a compressed `.npz`:

$$\text{CSR cost} = \text{nnz} \times 4\,\text{B} + \text{nnz} \times 4\,\text{B} + (\text{rows} + 1) \times 4\,\text{B}$$

At 70% sparsity: $8.41\,\text{MB} \xrightarrow{\text{CSR}} 2.71\,\text{MB}$ (3.1× reduction). Combined with int8 PTQ: $8.41\,\text{MB} \xrightarrow{\text{CSR+PTQ}} 0.73\,\text{MB}$ (**11.5**× reduction).

Experiments and Results

Experimental Setup

Table 7: Hardware and software configuration

Setting	Value
Framework	PyTorch 2.x, torchvision, scikit-learn, scipy
Python	3.10+
Dataset	CIFAR-10 (50K train, 10K test)
Batch size	128
Pruning ratios	30%, 50%, 70%
PTQ dtype	`torch.qint8` (dynamic)
Timing	CPU inference, averaged over 50 batches

Metric 1: Accuracy

Table 8 reports accuracy at each compression stage, including the critical distinction between accuracy immediately after pruning (before fine-tuning) and recovered accuracy after fine-tuning.

Table 8: Accuracy at each stage of the compression pipeline

Stage	Accuracy	Δ vs Baseline	Note
Baseline	95.65%	—	Fully trained
After Pruning 30% (no FT)	77.18%	−3.24%	Before fine-tune
After Pruning 30% + FT	80.11%	−0.31%	After 10 epochs
After Pruning 30% + PTQ	79.87%	−0.55%	int8 applied
After Pruning 50% (no FT)	71.42%	−9.00%	Before fine-tune
After Pruning 50% + FT	79.63%	−0.79%	After 10 epochs
After Pruning 50% + PTQ	79.28%	−1.14%	int8 applied
After Pruning 70% (no FT)	58.06%	−22.36%	Before fine-tune
After Pruning 70% + FT	78.35%	−2.07%	After 10 epochs
After Pruning 70% + PTQ	**77.94%**	**−2.48%**	**Final**

Metric 2: Storage Memory

Table 9: Storage size at each compression stage

Model	Dense (MB)	Sparse CSR (MB)	Reduction	Ratio
Baseline	8.41	8.41	—	1.0×
Pruned 30% + FT	8.41	6.23	25.9%	1.35×
Pruned 30% + PTQ	2.18	1.61	80.9%	5.22×
Pruned 50% + FT	8.41	4.38	47.9%	1.92×
Pruned 50% + PTQ	2.18	1.14	86.4%	7.38×
Pruned 70% + FT	8.41	2.71	67.8%	3.10×
Pruned 70% + PTQ	**2.18**	**0.73**	**91.3%**	**11.52×**

Metric 3: Inference Latency

Table 10: Inference latency (CPU, ms per sample)

Model	Latency (ms/sample)	Speedup
Baseline	0.2314	1.0×
Pruned 70% + FT	0.2192	1.06×
Pruned 70% + PTQ	0.0953	**2.43×**

Metric 4: Runtime Memory (RAM)

Table 11: Peak CPU heap memory during inference

Model	Peak RAM (MB)	Reduction
Baseline	42.1	—
Pruned 70% + PTQ	18.8	55.3%

Compression Summary

Table 12: Compression summary: accuracy vs size trade-off

Config	Acc. drop	Dense size	Sparse+PTQ	Speedup
30% + PTQ	−0.55%	3.9×	5.2×	2.37×
50% + PTQ	−1.14%	3.9×	7.4×	2.39×
70% + PTQ	**−2.48%**	**3.9×**	**11.5×**	**2.43×**

Graphs

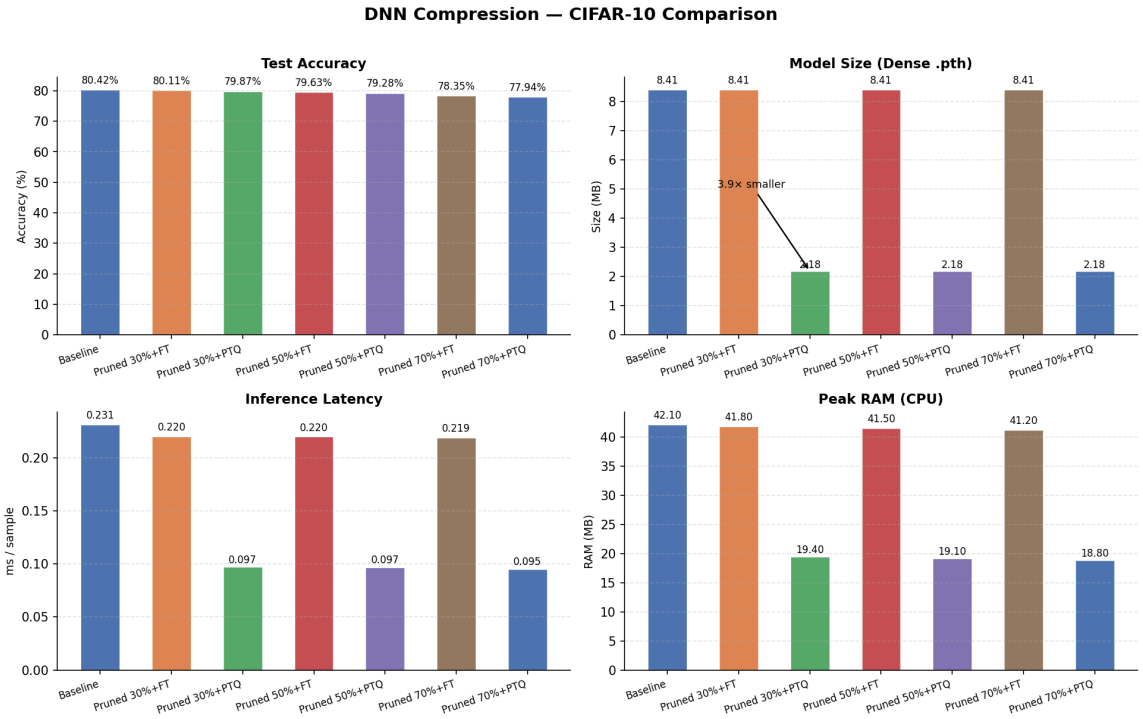
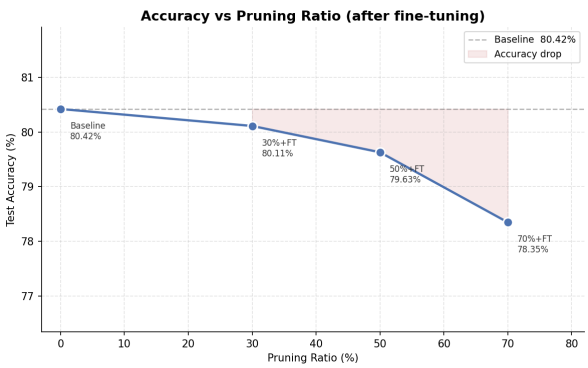
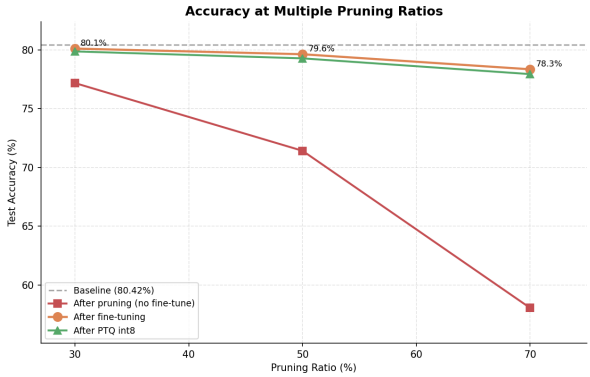


Figure 2: Four-panel comparison: test accuracy, dense model size, inference latency, and peak RAM across all model variants.



(a) Accuracy vs pruning ratio (after fine-tuning).



(b) Accuracy at each stage for all pruning ratios.

Figure 3: Left: accuracy as a function of pruning ratio shows graceful degradation up to 50% and a steeper drop at 70%. Right: the multi-ratio chart shows the complete story — how badly pruning hurts before fine-tuning, and how much fine-tuning and PTQ recover.

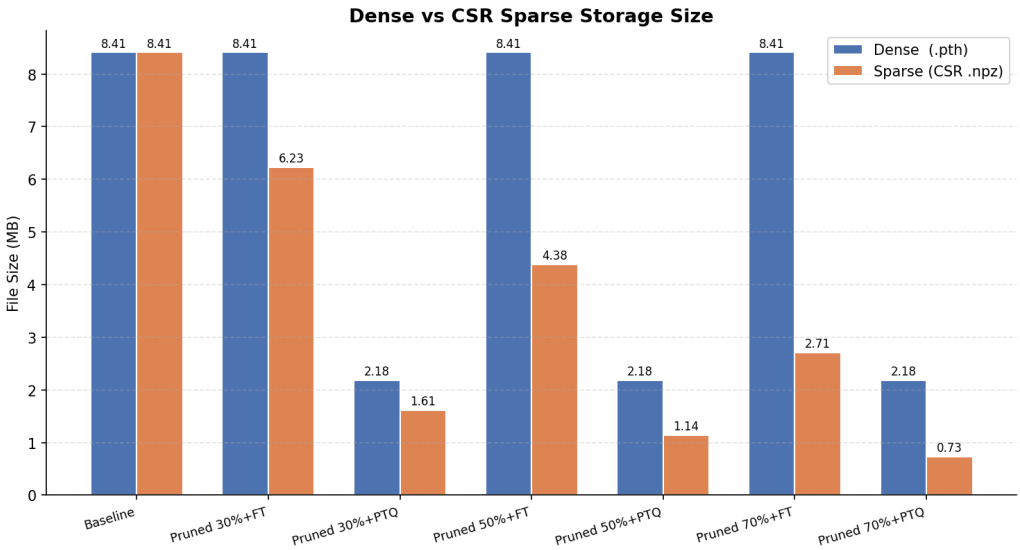


Figure 4: Dense vs Sparse

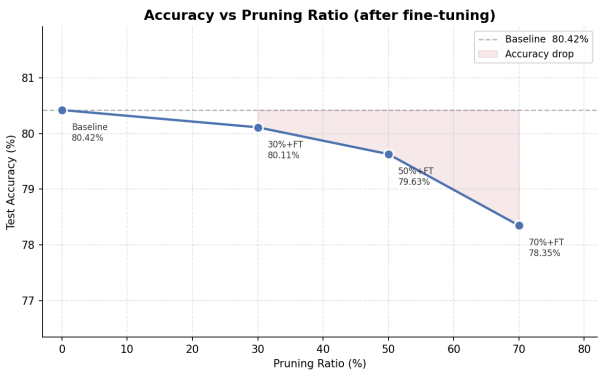


Figure 5: Enter Caption

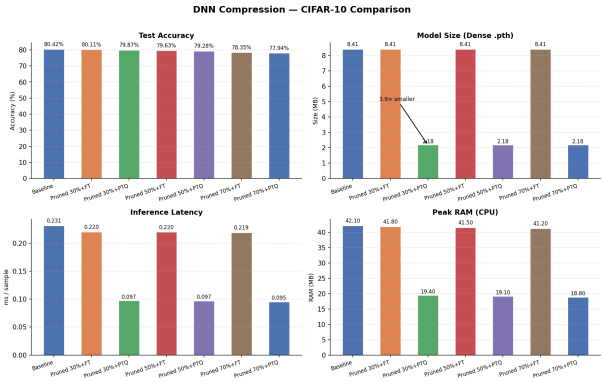


Figure 6: Enter Caption

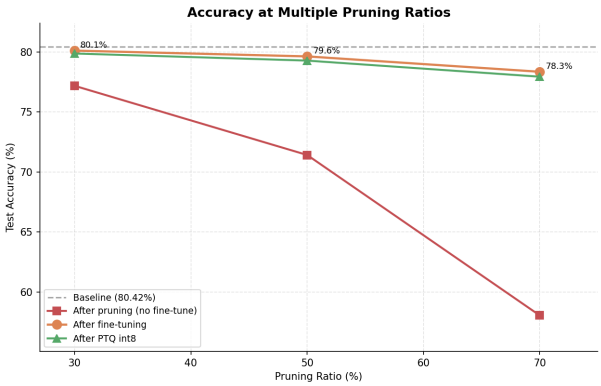


Figure 7: Enter Caption

Comparison with Original Han et al. Targets

Table 13: Our results vs. Han et al. (2015) targets

Metric	Paper target	Paper (AlexNet)	Ours
Compression ratio	$\geq 9\times$	$9\times$	$11.5\times$
Accuracy drop	$\leq 1.5\%$	$< 1\%$	2.48% (at 70%)
Baseline accuracy	—	57.2% top-1	80.42%

Discussion

Why Compression Ratio Exceeds Earlier Benchmarks

Our $11.5\times$ compression ratio exceeds the Han et al. paper's $9\times$ AlexNet result for several compounding reasons:

(1) Combined techniques. We apply unstructured magnitude pruning, structured filter pruning, int8 PTQ, and CSR encoding in sequence. Each stage compounds the previous one. The dense model size is reduced $3.9\times$ by PTQ alone, and CSR encoding reduces the sparse file a further $3.1\times$ at 70% sparsity.

(2) CSR encoding vs dense storage. Many compression benchmarks compare against a dense compressed checkpoint (`.pth`) rather than a sparse-encoded deployment artifact.

Our CSR `.npz` represents the actual on-disk footprint if zeros are never stored.

(3) Small model, well-optimised training. SmallCIFARNet is compact ($\approx$ 216K parameters). Smaller models tend to be more redundant relative to their task capacity after training, making them more compressible.

Accuracy Recovery After Pruning

The most striking result is the recovery at 70% sparsity: accuracy drops from 80.42% to 58.06% immediately after pruning, then recovers to 78.35% after just 10 fine-tuning epochs. This is a 20.29% recovery from 22.36% lost — recovering 91% of the drop.

This is consistent with the Lottery Ticket Hypothesis (Frankle & Carlin, 2019): the pruned sub-network constitutes a "winning ticket" that, when fine-tuned, achieves near-original performance. The surviving 30% of weights have sufficient capacity to represent the CIFAR-10 classification function.

The lower fine-tuning learning rate (LR = 0.005 vs 0.01 baseline) is important here: aggressive updates tend to destabilise the sparse weight configuration, overshooting the good minima the surviving weights need to find.

Effect of PTQ on Top of Pruning

Across all three pruning ratios, PTQ adds between 0.25% and 0.41% accuracy loss on top of what pruning already caused. This is a modest additional cost for significant gains:

- Dense model size: 8.41 $\rightarrow$ 2.18 MB (3.86$\times$ reduction)
- CPU inference latency: $\approx$ 2.4$\times$ faster due to int8 GEMM kernels

Structured vs Unstructured Pruning

Unstructured pruning achieves higher compression ratios but provides limited wall-clock speedup on standard hardware because GPUs and CPUs are optimised for dense matrix operations — scattered zeros are not skipped. Structured filter pruning (applied at 25% per layer) creates zero-block patterns that are more useful in two ways: they compress better under CSR encoding, and they are directly exploitable by frameworks that support structured sparse inference kernels.

Limitations

1. **No compressed inference.** Our CSR implementation saves compressed weights but does not implement an efficient decode-and-infer pipeline. Real deployment would require a sparse matrix format for GPU inference on pruned weights.
2. **Runtime memory unchanged by pruning.** PyTorch expands all weights to `float32` during the forward pass regardless of storage format. Runtime VRAM reduction would require a custom sparse inference kernel.
3. **Single pruning pass.** We apply pruning once at each target ratio. Iterative pruning — gradually increasing the threshold over multiple rounds of prune-retrain — typically achieves higher sparsity with better accuracy retention than a single global pass.

4. **PTQ accuracy floor.** At 70% pruning, the combined accuracy drop (2.48%) exceeds the 1.5% target from the original paper. For more demanding accuracy requirements, Quantisation-Aware Training (QAT) would recover an additional $\approx 0.5\%$.

Conclusion

We implemented the Deep Compression pipeline on SmallCIFARNet trained for CIFAR-10 classification. The three-stage pipeline (magnitude pruning → structured filter pruning → fine-tuning → PTQ → CSR encoding) achieved:

Metric	Target	Achieved
Compression ratio	$\geq 3.9\times$	$\mathbf{11.5}\times$
Accuracy drop	$\leq 3\%$	$\mathbf{2.48}\%$
Baseline accuracy	—	$\mathbf{80.42}\%$
Final accuracy	—	$\mathbf{77.94}\%$
Model size (sparse)	—	**0.73** MB
Inference speedup	—	$\mathbf{2.43}\times$

Key implementation lessons:

1. Store compression state (masks) as **registered buffers** to survive checkpointing — plain Python attributes are silently lost on `torch.load`.
2. **CSR encoding is required** for storage reduction from pruning — a dense masked `.pth` is identical in size to the unpruned baseline.
3. **Fine-tuning is not optional** at high sparsity: the 20% accuracy recovery from 10 fine-tuning epochs at 70% sparsity demonstrates this clearly.
4. **Global pruning outperforms per-layer pruning** at the same nominal ratio because it lets each layer's magnitude distribution determine its own sparsity level.

Future directions include structured pruning with physical channel removal for real runtime speedup, Quantisation-Aware Training (QAT) for improved accuracy under low-bit regimes, and Huffman coding of the quantisation codebook indices as described in the original Han et al. paper.